

LAB 12. XÂY DỰNG VÀ KIỂM THỬ RESTFUL API (CRUD) BẰNG POSTMAN

Bối cảnh: Frontend team đang chuẩn bị xây dựng một ứng dụng Mobile quản lý Thư viện. Họ yêu cầu Backend team cung cấp một bộ API chuẩn RESTful để thực hiện các thao tác Thêm, Xem, Sửa, Xóa danh sách các cuốn sách (Books).

Mục tiêu: Tự tay xây dựng hoàn chỉnh luồng API từ Database -> Model -> Resource -> Controller -> Route và sử dụng công cụ Postman để đóng vai trò "Mobile App" gửi request lên kiểm thử.

PHẦN 1: XÂY DỰNG HỆ THỐNG API (BACKEND)

Bước 1. Khởi tạo Model và Migration: Tạo bảng dữ liệu và khai báo Model.

Mở Terminal và chạy lệnh để tạo Model Book kèm file Migration:

```
php artisan make:model Book -m
```

1. Cấu trúc bảng:

Mở file database/migrations/xxxx_create_books_table.php, bổ sung vào hàm up():

```
public function up(): void
{
    Schema::create('books', function (Blueprint $table) {
        $table->id();
        $table->string('title'); // Tên sách
        $table->string('author'); // Tác giả
        $table->integer('price'); // Giá bán
        $table->boolean('is_published')->default(true); // Trạng thái
        $table->timestamps();
    });
}
```

2. Cấu hình Model:

Mở file app/Models/Book.php, cho phép lưu dữ liệu hàng loạt:

```
class Book extends Model
{
    protected $fillable = ['title', 'author', 'price', 'is_published'];
}
```

3. Chạy Migration:

Đẩy bảng lên CSDL bằng lệnh php artisan migrate.

```
php artisan migrate
```

Bước 2. Tạo API Resource: Định dạng lại dữ liệu trả về cho Mobile App.

Chạy lệnh Terminal để tạo "màng lọc" định dạng JSON:

```
php artisan make:resource BookResource
```

Mở app/Http/Resources/BookResource.php và cấu trúc lại dữ liệu (Viết hóa các key để Frontend dễ đọc):

```
public function toArray(Request $request): array
{
    return [
        'ma_sach' => $this->id,
        'tieu_de' => $this->title,
        'tac_gia' => $this->author,
        'gia_ban' => $this->price,
        'trang_thai' => $this->is_published ? 'Đang bán' : 'Ngừng bán',
        'ngay_nhap' => $this->created_at->format('d/m/Y'),
    ];
}
```

Bước 3. Tạo Controller và Đăng ký Route: Viết logic CRUD và mở cửa API.

1. Tạo Controller:

Chạy lệnh tạo Controller chuyên dụng cho API (hệ số --api giúp bỏ qua các hàm trả về view html):

```
php artisan make:controller Api/BookController --api
```

2. Đăng ký Route:

Mở file routes/api.php (nếu chưa có, chạy lệnh php artisan install:api) và thêm:

```
use App\Http\Controllers\Api\BookController;
use Illuminate\Support\Facades\Route;

// Đăng ký toàn bộ 5 thao tác CRUD cho đường dẫn /api/books
Route::apiResource('books', BookController::class);
```

Bước 4. Viết logic xử lý trong Controller: Hoàn thiện 5 thao tác chuẩn RESTful.

Mở file `app/Http/Controllers/Api/BookController.php` và hoàn thiện code:

```
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Book;
use App\Http\Resources\BookResource;
use Illuminate\Http\Request;

class BookController extends Controller
{
    // 1. Lấy danh sách (GET)
    public function index()
    {
        return BookResource::collection(Book::all());
    }

    // 2. Thêm mới (POST)
    public function store(Request $request)
    {
        $validatedData = $request->validate([
            'title' => 'required|string|max:255',
            'author' => 'required|string|max:255',
            'price' => 'required|integer|min:0'
        ]);

        $book = Book::create($validatedData);

        return response()->json([
            'message' => 'Thêm sách thành công!',
            'data' => new BookResource($book)
        ], 201);
    }

    // 3. Lấy chi tiết (GET)
    public function show(string $id)
    {
        $book = Book::find($id);

        if (!$book) return response()->json(['message' => 'Không tìm thấy
```

```
sách'], 404);

    return new BookResource($book);
}

// 4. Cập nhật (PUT)
public function update(Request $request, string $id)
{
    $book = Book::find($id);
    if (!$book) return response()->json(['message' => 'Không tìm thấy sách'], 404);

    $book->update($request->all());

    return response()->json([
        'message' => 'Cập nhật thành công!',
        'data' => new BookResource($book)
    ], 200);
}

// 5. Xóa (DELETE)
public function destroy(string $id)
{
    $book = Book::find($id);
    if (!$book) return response()->json(['message' => 'Không tìm thấy sách'], 404);

    $book->delete();
    return response()->json(['message' => 'Xóa sách thành công!'],
200);
}
}
```

PHẦN 2: KIỂM THỬ API VỚI POSTMAN

Đảm bảo server Laravel đang chạy bằng lệnh php artisan serve trên port 8000. Mở phần mềm Postman và thực hiện các kịch bản sau.

- Ở mọi Request, bạn phải chuyển sang tab **Headers** và thêm:
 - **Key:** Accept
 - **Value:** application/json

Điều này báo cho Laravel biết bạn là Client API, nếu lỗi Validate xảy ra, nó sẽ trả về mã lỗi JSON thay vì báo lỗi HTML.

- **Test 1: Tạo mới sách (CREATE)**

- **Method:** POST | **URL:** http://localhost:8000/api/books
- **Body -> raw -> JSON:**

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 250000
}
```

- **Kỳ vọng:** Trả về mã 201 Created kèm thông báo thành công.

- **Test 2: Lấy danh sách (READ)**

- **Method:** GET | **URL:** http://localhost:8000/api/books
- **Kỳ vọng:** Trả về danh sách sách đã được dịch sang tiếng Việt (nhờ BookResource).

- **Test 3: Cập nhật sách (UPDATE)**

- **Method:** PUT | **URL:** http://localhost:8000/api/books/1 (đổi id tương ứng)
- **Body -> raw -> JSON:** {"price": 300000, "is_published": false}
- **Kỳ vọng:** Trả về mã 200 OK báo cập nhật thành công.

- **Test 4: Xóa sách (DELETE)**

- **Method:** DELETE | **URL:** http://localhost:8000/api/books/1
- **Kỳ vọng:** Trả về thông báo "Xóa sách thành công!". Gọi lại Test 2 sẽ không thấy sách này nữa.

BÀI TẬP MỞ RỘNG

Mở rộng 1: Lọc dữ liệu & Phân trang (Pagination)

Gọi Book::all() với 10,000 cuốn sách sẽ làm sập ứng dụng. Hãy sửa hàm index() trong BookController:

```
public function index(Request $request)
{
```

```
$query = Book::query();

if ($request->has('search')) {
    $query->where('title', 'LIKE', '%' . $request->search . '%');
}

$books = $query->paginate(5); // 5 cuốn 1 trang
return BookResource::collection($books);
}
```

Yêu cầu nộp: Chụp màn hình gọi API /api/books?search=Code, kết quả trả về phải chứa các key phân trang (meta, links).

Mở rộng 2: Bảo mật bằng Sanctum

Chỉ những "Thủ thư" đã đăng nhập mới được thao tác thay đổi dữ liệu.

1. Tạo hàm /api/login để cấp phát Token (theo lý thuyết đã học).
2. Mở routes/api.php, nhóm các Route POST, PUT, DELETE vào trong Route::middleware('auth:sanctum').

Yêu cầu nộp:

- Ảnh 1: Gọi POST thêm sách không có Token -> Bị chặn (Lỗi 401).
- Ảnh 2: Thêm Token vào thẻ Authorization (Bearer) -> Thêm thành công.

Mở rộng 3: Upload File qua API

Mobile App muốn gửi kèm ảnh bìa sách.

1. Tạo cột cover_image (string, nullable) trong bảng books.
2. Tại Postman: **Không dùng** tab raw -> JSON. Phải chuyển sang **Body -> form-data**, cấu hình Key cover_image kiểu **File** và chọn ảnh từ máy tính.

Yêu cầu nộp: Ảnh Postman đang upload file thành công.

BIỂU MẪU NỘP BÀI LAB 13

Copy phần này vào Word, lưu tên file MSSV_HoTen_Lab13.docx

THỰC HÀNH LAB 13: XÂY DỰNG API CRUD

- **Họ và tên:**
- **MSSV:**

Hình 1: Test Thêm mới (POST)

Yêu cầu: Cột author phải điền chính xác Họ và tên sinh viên. Chụp rõ giao diện báo mã 201.

[Chèn hình 1]

Hình 2: Test Lấy danh sách (GET)

Yêu cầu: Request GET trả về dữ liệu chuẩn hóa có các key tiếng Việt: ma_sach, tieu_de...

[Chèn hình 2]

Hình 3: Xử lý lỗi Validate (422 Unprocessable Entity)

Yêu cầu: Gửi lệnh POST nhưng cố tình bỏ trống thuộc tính title. Header bắt buộc có Accept: application/json. Chụp giao diện báo lỗi.

[Chèn hình 3]